

Content Guide



Introduction to the Jupiter System

Powering No One Lives Forever™ 2, TRON® 2.0 and numerous other games produced by developers in North America, Asia, and Europe, Jupiter is a comprehensive set of tools and technologies that gives developers wanting to push the envelope the power to create high performance, immersive games for the PC and Xbox.

Jupiter handles model animation and physics, as well as rendering, special effects, networking, input management, audio, music, and so on. Jupiter is unique in that the game code and engine code are completely separate entities. This is what gives Jupiter much of its flexibility.

Jupiter includes the following features and advantages:

- Complete game code for No One Lives Forever™ 2.
- A new world renderer, which draws 3D graphics in larger chunks, enabling the system to work faster and more efficiently. With the new renderer, developers using Jupiter may create detailed, high poly environments.
- Enables developers to create large outdoor environments. It also features gouraud shading, which gives developers the ability to create shadows instead of using lightmaps, which will render worlds faster and more efficiently.

Development Tasks

When developing a 3D game application there are many tasks that must be completed. Planning, scheduling, and designing are well beyond the scope of this document. For games using Jupiter, the tasks can be assembled into the following groups:

- Design levels: Use the DEdit tool to design levels.
- Create models: Use Max or Maya along with the ModelEdit tool to create models.
- Create sound and music: Use a sound creation tool of your choice and the DirectMusic tools to create sound and music files.
- Create textures: Use a tool that exports TGA files, and convert the textures to .DTX files in DEdit.
- Write engine code: Jupiter provides full engine functionality.
- Write and/or modify game code: The task for your engineers.

The first four tasks are the responsibility of artists and level designers. The tools associated with them are described in the *Using the Tools* section of this document.

The fifth task, writing engine code, has already been done. Jupiter saves the game engineer the time and effort required to write the engine code upon which your game is based. (However, you may modify the engine code if desired.)

The last task, writing game code, is what remains for game programmers. Using Jupiter game code to provide basic functionality, you must write game code that handles the specific operations required by your game. See the *Programming* section of this document.

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



System Requirements

- [Windows Requirements](#)
- [Required Environment Variables](#)

Windows Requirements

All Jupiter Editions

- Windows XP or Windows 2000 with SP2 or higher
- Microsoft Visual C++ .NET 2003 (7.1) or Microsoft Visual C++ .NET 2002 (7.0). Because of DirectX9, Microsoft Visual C++ 6.0 is no longer supported.
- Microsoft® DirectX® 9.0c SDK Retail Version
 - Maya 6.0, 5.0, 4.5, 4.0, or 3.0
- 3D Studio Max 6.0 or 5.0 with Character Studio 4, 3D Studio Max 4.0 with Character Studio 3.1.2, or 3D Studio Max 3.0 with Character Studio 3.0.2. Character Studio 3.0, 3.0.1, 3.1, and 3.1.1 are not supported.
- Bink Video for Windows is optional for video playback. Bink is available from RAD Game Tools at

<http://www.radgametools.com/>

Enterprise Edition Only

- SDKs for Maya 6.0, 5.0, 4.5, 4.0, or 3.0
- SDKs for 3D Studio Max 6.0 or 5.0 with Character Studio 4, 3D Studio Max 4.0 with Character Studio 3.1.2, or 3D Studio Max 3.0 with Character Studio 3.0.2.

Required Environment Variables

In order to compile the engine, tools, and samples, you must create the following environment variables:

<u>Environment Variable Name</u>	<u>Value</u>
GAME_MAIN_DIR	Install path to game executable directory (Ex. C:\TouchdownEntertainment\Jupiter_Ent\Development\TO2)
GAME_REZ_DIR	Install path to game resource directory (Ex. C:\TouchdownEntertainment\Jupiter_Ent\Development\TO2\Game)
GAME_TOOLS_DIR	Install path to Jupiter PC tools directory (Ex. C:\TouchdownEntertainment\Jupiter_Ent\Tools)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

Content Guide



Implementing Your Game Using Jupiter

The Jupiter System arms your developers with the tools and code base to enable your entire game team to focus on the creation of compelling game content. Instead of spending their time designing and coding a renderer or physics system, for example, they can concentrate on supporting level designers and artists.

This section provides some suggestions for implementing a game using Jupiter and exploiting the benefits of having developers with (relatively) more time on their hands. This document assumes that you have already completed a game specification and have a design document very near completion. Armed with these imperatives, your team can roll up their sleeves and start working on the game.

Note: While you can use the No One Lives Forever 2 objects and code provided with Jupiter, the art resources are included as samples only and cannot be included in any released games. TO2 textures, models, levels, and animations are the property of Monolith Productions, and may not be included in any released product.

This section contains the following suggestions to aid you in getting started implementing your game using the Jupiter system:

- [Learn the Jupiter System](#)
- [Review your Design Documentation](#)
- [Identify your Development Tasks](#)
- [Modify your Object Properties](#)
- [Create your New Levels](#)
- [Develop your Artwork](#)

Learn the Jupiter System

The first thing that your entire game team must do is sufficiently acquaint themselves with Jupiter.

For developers, this means, at the very least, learning how the game objects work so that they can modify and/or duplicate them. If you are adding significant functionality to the Jupiter code base, then they will also need to familiarize themselves with the sections of code to be modified. Examining the TO2 game code is a perfect way to learn how Jupiter works. The engine code includes comments for specific classes, methods, and so on. The Programming section of this document provides many overviews for the various Jupiter subsystems.

Level designers must acquaint themselves with the tools and many game objects and level design issues unique to Jupiter. The DEdit and ModelEdit tools give your artists significant powers in designing levels and models. Reviewing the TO2 levels provided with Jupiter is ideal for learning about Jupiter objects. Level designers can also peruse the objects and properties listings in DEdit.

[Top](#)

Review Your Design Documentation

After your developers and level designers have familiarized themselves with what Jupiter has to offer, they should spend some time comparing the resources of Jupiter to the demands of your design document. That is, does Jupiter provide all the objects and functionality required by your game? You should identify all such differences and prioritize them. After completing this task, you should have a good idea of the work load for your developers.

[Top](#)

Identify Your Development Tasks

Your developers, freed from the burden of coding the game engine, are going to have a lot of time to support the needs of your level designers. The following list describes some of the common tasks your developers need to complete:

- **Reviewing Code and Objects**

As stated previously, it is imperative that your developers learn the Jupiter code so that they can efficiently modify or add to it. For more information about the Jupiter code, see the Jupiter API Reference.

- **Setting Up the Game Project**

You will need a project in which the level designers and artists can test their content. The easiest way to set up your game project is to use the TO2 game as a base by renaming TO2.dsw, and then modifying the project to suit your own requirements.

- **Altering Existing Objects**

Obviously, your game will likely require objects uniquely suited to your own game, and radically different than those provided by the TO2 objects. It is often possible to modify the existing TO2 objects to suit your needs rather than putting forth the effort to create entirely new objects.

- **Creating New Objects**

Your game may demand such a unique object that none of the existing TO2 objects can be easily altered to fulfill the required functionality. In this case, your developers will need to create a new object. The existing objects, however, can still provide very good examples for this endeavor.

- **Adding or Altering Code**

If your game has extreme or unique game play elements your developers may need to enhance the Jupiter code base.

- **Purging Unneeded Code**

Near the end of the development cycle, your developers should sweep the game project for unneeded code. If you used the TO2 workspace as a starting point, there may be extraneous code that your game does not utilize. To streamline your game code, some effort should be expended in removing this code.

Be sure to examine the code carefully before deleting it to make sure it has no dependencies and will not result in a bug.

- **Designing Levels**

Your level designers will have more access to receptive developers thanks to the game engine Jupiter provides. This should translate into quicker turnaround on functionality requests and improved content.

Level designers will continue to design levels as usual, but they should pay attention to these Jupiter-specific issues:

- **Learning the Tools**

The DEdit and ModelEdit tools provided with Jupiter are used to create levels and prepare models for inclusion in those levels, respectively. Unless they know these tools, designers will find their efforts hampered.

- **Learning the Objects**

Jupiter includes many TO2 objects that you can leverage in your game. At the beginning of your game development cycle, designers should determine which of these objects your game will use. They should also help identify which of your game design requirements are not fulfilled by Jupiter and inform the developers of them.

• Reviewing the Levels

A good learning experience for your level designers is to traverse the TO2 levels included with Jupiter. They should keep an eye out for issues unique to your game design and critically analyze your games levels to determine how to accomplish your needs using Jupiter. It would be a good idea to have a developer present during this review process so they can provide real-time input on the feasibility of various requests. After you start the game, click Single Player and then Custom Level to choose from the levels included with Jupiter.

[Top](#)

Modify Your Object Properties

Many of the TO2 objects will closely fulfill the requirements of your game. Often, all you must do is alter the properties of an object to make it comply with the demands of your game. Of course, you will also have to reset the textures and models for your objects to utilize your art assets instead of the proprietary TO2 assets.

[Top](#)

Create Your New Levels

Using DEdit and the modified objects, your level designers must create new levels for your game.

[Top](#)

Develop Your Artwork

Artists must merely comply with a few Jupiter requirements when creating their textures and models. They must also use the ModelEdit tool to prepare the model for inclusion in the Jupiter game.

When creating a model in 3D Studio MAX or Maya, follow these high-level steps:

1. Create a sphere.
2. Create a joint.
3. Bind the skin.
4. Export.

Note: Use 32 bit .TGA files for textures in the Jupiter system.

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



Components

The various Jupiter components are located in the following subdirectories of \TouchdownEntertainment\Jupiter:

\Development\TO2	Binary No One Lives Forever™ 2 game and resources
\Docs	Documentation
\Engine\Built	Compiled engine libraries and executables
\Engine\ClientFx	ClientFx source code
\Engine\Libs	Engine libraries
\Engine\Runtime	Engine source code
\Engine\sdk	SDK header files
\Engine\Tools	Tool source code
\Game	No One Lives Forever™ 2 source code
\Libs	Library source code
\Samples	Sample source code

\Samples\bin
\Tools

Compiled engine binaries
Tool binaries

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

Content Guide



Building and Running Jupiter

- [Building from the Command Line](#)
 - [Building from Visual Studio](#)
 - [Running the Samples](#)
- [Running No One Lives Forever™ 2](#)
 - [Debugging the Samples](#)
- [Launching the Game Within DEdit](#)

Building from the Command Line

To build the engine from the command line:

1. Open a DOS window.
2. Switch to the \TouchdownEntertainment\Jupiter\Engine\Runtime\ directory.
3. Run the **build.bat** file for Visual Studio .NET 2003 (v7.1) or the **build2002.bat** file for Visual Studio .NET 2002 (v7).
4. Open the build.log file to view the output.

You can also open the build.bat file from an explorer window, but you won't be able to review the output after the build process has completed.

To build the engine tools from the command line:

1. Open a DOS window.
2. Switch to the \TouchdownEntertainment\Jupiter\Engine\Tools\ directory.
3. Run the **build.bat** file for Visual Studio .NET 2003 (v7.1) or the **build2002.bat** file for Visual Studio .NET 2002 (v7).
4. Open the build.log file to view the output.

In order for the exporter plugins to build, you will need to copy over the Max and/or Maya SDK files. See the readme files in the \Engine\Tools\Plugins\ sub-folders for more information.

To build the samples from the command line:

1. Open a DOS window.
2. Switch to the \TouchdownEntertainment\Jupiter\Samples\ directory.
3. Run the **build.bat** file for Visual Studio .NET 2003 (v7.1) or the **build2002.bat** file for Visual Studio .NET 2002 (v7).
4. Open the build.log file to view the output.

To build the No One Lives Forever™ 2 game code from the command line:

1. Open a DOS window.
2. Switch to the \TouchdownEntertainment\Jupiter\Game\ directory.
3. Run the **build.bat** file for Visual Studio .NET 2003 (v7.1) or the **build2002.bat** file for .NET 2002 (v7).
4. Open the build.log file to view the output.

Building from Visual Studio

Visual Studio .NET 2003 (v7.1) Instructions:

To build the engine code from Visual Studio .NET 2003 (v7.1):

1. Right click on the \TouchdownEntertainment\Jupiter\Engine\Runtime\Engine.sln file and select **Open With Visual Studio .NET 2003** (you may also double click on the sln file).
2. If any **Source Control** dialogs appear, click **No** or **Cancel** to dismiss them.
3. Select **Rebuild Solution** from the **Build** menu.

To build the engine tools code from Visual Studio .NET 2003 (v7.1):

1. Right click on the \TouchdownEntertainment\Jupiter\Engine\Tools\Tools.sln file and select **Open With Visual Studio .NET 2003** (you may also double click on the sln file).
2. If any **Source Control** dialogs appear, click **No** or **Cancel** to dismiss them.
3. Select **Rebuild Solution** from the **Build** menu.

To build the samples from Visual Studio .NET 2003 (v7.1):

1. Right click on the project solution file (*.SLN) in the sample folder (Ex. \TouchdownEntertainment\Jupiter\Samples\base\samplebase\samplebase.sln) and select **Open With Visual Studio .NET 2003** (you may also double click on the sln file).
2. If any **Source Control** dialogs appear, click **No** or **Cancel** to dismiss them.
3. Select **Rebuild Solution** from the **Build** menu.

To build the No One Lives Forever™ 2 game code from Visual Studio .NET 2003 (v7.1):

1. Right click on the \TouchdownEntertainment\Jupiter\Game\TO2.sln file and select **Open With Visual Studio .NET 2003** (you may also double click on the sln file).
2. If any **Source Control** dialogs appear, click **No** or **Cancel** to dismiss them.
3. Select **Rebuild Solution** from the **Build** menu.

Visual Studio .NET 2002 (v7) Instructions:To build the engine code from Visual Studio .NET 2002 (v7):

1. Right click on the \TouchdownEntertainment\Jupiter\Engine\Runtime\Engine2002.sln file and select **Open With Visual Studio .NET 2002** (you may also double click on the sln file).
2. If any **Source Control** dialogs appear, click **No** or **Cancel** to dismiss them.
3. Select **Rebuild Solution** from the **Build** menu.

To build the engine tools code from Visual Studio .NET 2002 (v7):

1. Right click on the \TouchdownEntertainment\Jupiter\Engine\Tools\Tools2002.sln file and select **Open With Visual Studio .NET 2002** (you may also double click on the sln file).
2. If any **Source Control** dialogs appear, click **No** or **Cancel** to dismiss them.
3. Select **Rebuild Solution** from the **Build** menu.

To build the samples from Visual Studio.NET 2002 (v7):

1. Right click on the project solution file (*2002.SLN) in the sample folder (Ex. \TouchdownEntertainment\Jupiter\Samples\base\samplebase\samplebase2002.sln). and select **Open With Visual Studio .NET 2002** (you may also double click on the sln file).
2. If any **Source Control** dialogs appear, click **No** or **Cancel** to dismiss them.
3. Select **Rebuild Solution** from the **Build** menu.

To build the No One Lives Forever™ 2 game code from Visual Studio .NET 2002 (v7):

1. Right click on the \TouchdownEntertainment\Jupiter\Game\TO22002.sln file and select **Open With Visual Studio .NET 2002** (you may also double click on the sln file).
2. If any **Source Control** dialogs appear, click **No** or **Cancel** to dismiss them.
3. Select **Rebuild Solution** from the **Build** menu.

Running the Samples**To run the samples from the command line:**

To run the samples from the command line, first build the sample and then execute the **run.bat** file for Visual Studio .NET 2003 (v7.1) or the **run2002.bat** file for Visual Studio .NET 2002 (v7) in the root directory of the sample. (Ex. \TouchdownEntertainment\Jupiter\Samples\base\samplebase\run.bat. This would run the Visual Studio .NET 2003 release build of the sample.)

Running No One Lives Forever™ 2

To run No One Lives Forever™ 2, you can run the game launcher from:

\TouchdownEntertainment\Jupiter\Development\TO2\NOLF2.exe

Debugging the Samples**To debug the samples from Visual Studio .NET 2003 (v7.1):**

To debug the samples using Visual Studio .NET 2003 (v7.1), follow the instructions in the previous section to build the engine. Next, run **copy_binaries.bat** in the \TouchdownEntertainment\Jupiter\Samples\bin folder. In Visual Studio .NET 2003, open the project properties for **cshell** and click the **Debugging** section and enter the following parameters:

Command:

Enter the path and filename for the lithtech executable:

```
\TouchdownEntertainment\Jupiter\Samples\bin\debug\lithtech.exe
```

Command Arguments:

Enter the following text:

```
-rez engine.rez -rez ..\..\sample_type\sample_dir\rez -  
config ..\..\sample_type\sample_dir\bin\autoexec.cfg
```

Note that "sample_type/sample_dir" is the directory where the sample exists. For example, the command arguments for the SampleBase sample would be:

```
-rez engine.rez -rez ..\..\base\samplebase\rez -  
config ..\..\base\samplebase\bin\autoexec.cfg
```

Working Directory:

Enter the path to the Samples bin\debug directory:

```
\TouchdownEntertainment\Jupiter\Samples\bin\debug
```

To debug the samples from Visual Studio .NET 2002 (v7):

To debug the samples using Visual Studio .NET 2002 (v7), follow the instructions in the previous section to build the engine. Next, run **copy_binaries2002.bat** in the \TouchdownEntertainment\Jupiter\Samples\bin folder. In Visual Studio .NET 2002, open the project properties for **cshell** and click the **Debugging** section and enter the following parameters:

Command:

Enter the path and filename for the lithtech executable:

```
\TouchdownEntertainment\Jupiter\Samples\bin\debug2002\lithtech.exe
```

Command Arguments:

Enter the following text:

```
-rez engine.rez -rez ..\..\sample_type\sample_dir\rez -  
config ..\..\sample_type\sample_dir\bin\autoexec.cfg
```

Note that "sample_type/sample_dir" is the directory where the sample exists. For example, the command arguments for the SampleBase sample would be:

```
-rez engine.rez -rez ..\..\base\samplebase\rez -  
config ..\..\base\samplebase\bin\autoexec.cfg
```

Working Directory:

Enter the path to the Samples bin\debug2002 directory:

```
\TouchdownEntertainment\Jupiter\Samples\bin\debug2002
```

Launching the Game Within DEdit

To launch No One Lives Forever™ 2 from DEdit:

1. Run DEdit.
2. Select **Edit** and then **Options**.
3. Select the **Run** tab.
4. Enter the following parameters:

Executable:

Enter the path and filename for the lithtech executable:

```
C:\TouchdownEntertainment\Jupiter\Development\T02\Lithtech.exe
```

Working directory:

Enter the path to the root of the NOLF2 game directory:
 C:\TouchdownEntertainment\Jupiter\Development\T02

Program arguments:

Enter the following text: `-rez engine.rez -rez ..\..\Game +runworld %WorldName%`

5. Select **OK**.
6. When you have a world open, you can now click the **Run World** button to launch the game.

To launch a sample from DEdit:

1. Run DEdit.
2. Select **Edit** and then **Options**.
3. Select the **Run** tab.
4. Enter the following parameters:

Executable:

Enter the path and filename for the littech executable:
 C:\TouchdownEntertainment\Samples\bin\Release\Littech.exe

Working directory:

Enter the path to the littech executable: C:\TouchdownEntertainment\Samples\bin\Release

Program arguments:

Enter the following text:

```
-rez engine.rez -rez ..\..\sample_type\sample_dir\rez -
config ..\..\sample_type\sample_dir\bin\autoexec.cfg +runworld %WorldName%
```

Note that "sample_type\sample_dir" is the directory where the sample exists. For example, the program arguments for the SampleBase sample would be:

```
-rez engine.rez -rez ..\..\base\samplebase\rez -
config ..\..\base\samplebase\bin\autoexec.cfg +runworld %WorldName%
```

5. Select **OK**.
6. When you have a world open, you can now click the **Run World** button to launch the game.

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



Content Plugins

- [Content Plugins Installation](#)
- [Third Party SDK Installation](#)

Content Plugins Installation

The Jupiter plugins for 3D Studio Max, Maya, and Photoshop should be installed to the following directories:

Maya 6.0 SDK

You will need to run "rehash" on the Maya command line to load the scripts correctly. Consult your Maya documentation "What's new in Maya6" Chapter 12 for details.

Plugin File

\tools\Plugins\Alias\Maya6.0\bin\plug-ins\image\IMFLithDTX.dll
 \tools\Plugins\Alias\Maya6.0\bin\plug-ins\MayaModelExport60.mll
 \tools\Plugins\Alias\Maya6.0\bin\plug-ins\MayaWorldExport60.mll

Install Path

\Program Files\Alias\Maya6.0\bin\plug-ins\image
 \Program Files\Alias\Maya6.0\bin\plug-ins

\tools\Plugins\Alias\Maya6.0\scripts\LithTechModelExportOptions.mel
 \tools\Plugins\Alias\Maya6.0\scripts\LithTechWorldExportOptions.mel
 \tools\Plugins\Alias\Maya6.0\scripts\LithTechWorldImportOptions.mel

\Program Files\Alias\Maya6.0\bin\plug-ins
 \My Documents\Alias\Maya6.0\scripts\
 \My Documents\Alias\Maya6.0\scripts\
 \My Documents\Alias\Maya6.0\scripts\

Maya 5.0 SDK

Plugin File

\tools\Plugins\AliasWavefront\Maya5.0\bin\plug-ins\image\IMFLithDTX.dll
 \tools\Plugins\AliasWavefront\Maya5.0\bin\plug-ins\MayaModelExport50.mll
 \tools\Plugins\AliasWavefront\Maya5.0\bin\plug-ins\MayaWorldExport50.mll
 \tools\Plugins\AliasWavefront\Maya5.0\scripts\LithTechModelExportOptions.mel
 \tools\Plugins\AliasWavefront\Maya5.0\scripts\LithTechWorldExportOptions.mel
 \tools\Plugins\AliasWavefront\Maya5.0\scripts\LithTechWorldImportOptions.mel

Install Path

\Program Files\AliasWavefront\Maya5.0\bin\plug-ins\image
 \Program Files\AliasWavefront\Maya5.0\bin\plug-ins
 \Program Files\AliasWavefront\Maya5.0\bin\plug-ins
 \Program Files\AliasWavefront\Maya5.0\scripts
 \Program Files\AliasWavefront\Maya5.0\scripts
 \Program Files\AliasWavefront\Maya5.0\scripts

Maya 4.5 SDK

Plugin File

\tools\Plugins\AliasWavefront\Maya4.5\bin\plug-ins\image\IMFLithDTX.dll
 \tools\Plugins\AliasWavefront\Maya4.5\bin\plug-ins\MayaModelExport45.mll
 \tools\Plugins\AliasWavefront\Maya4.5\bin\plug-ins\MayaWorldExport45.mll
 \tools\Plugins\AliasWavefront\Maya4.5\scripts\LithTechModelExportOptions.mel
 \tools\Plugins\AliasWavefront\Maya4.5\scripts\LithTechWorldExportOptions.mel
 \tools\Plugins\AliasWavefront\Maya4.5\scripts\LithTechWorldImportOptions.mel

Install Path

\Program Files\AliasWavefront\Maya4.5\bin\plug-ins\image
 \Program Files\AliasWavefront\Maya4.5\bin\plug-ins
 \Program Files\AliasWavefront\Maya4.5\bin\plug-ins
 \Program Files\AliasWavefront\Maya4.5\scripts
 \Program Files\AliasWavefront\Maya4.5\scripts
 \Program Files\AliasWavefront\Maya4.5\scripts

Maya 4.0 SDK

Plugin File

\tools\Plugins\AliasWavefront\Maya4.0\bin\plug-ins\image\IMFLithDTX.dll
 \tools\Plugins\AliasWavefront\Maya4.0\bin\plug-ins\MayaModelExport40.mll
 \tools\Plugins\AliasWavefront\Maya4.0\bin\plug-ins\MayaWorldExport40.mll
 \tools\Plugins\AliasWavefront\Maya4.0\scripts\LithTechModelExportOptions.mel
 \tools\Plugins\AliasWavefront\Maya4.0\scripts\LithTechWorldExportOptions.mel
 \tools\Plugins\AliasWavefront\Maya4.0\scripts\LithTechWorldImportOptions.mel

Install Path

\Program Files\AliasWavefront\Maya4.0\bin\plug-ins\image
 \Program Files\AliasWavefront\Maya4.0\bin\plug-ins
 \Program Files\AliasWavefront\Maya4.0\bin\plug-ins
 \Program Files\AliasWavefront\Maya4.0\scripts
 \Program Files\AliasWavefront\Maya4.0\scripts
 \Program Files\AliasWavefront\Maya4.0\scripts

Maya 3.0 SDK

Plugin File

\tools\Plugins\AliasWavefront\Maya3.0\bin\plug-ins\image\IMFLithDTX.dll
 \tools\Plugins\AliasWavefront\Maya3.0\bin\plug-ins\MayaModelExport30.mll
 \tools\Plugins\AliasWavefront\Maya3.0\bin\plug-

Install Path

\Program Files\AliasWavefront\Maya3.0\bin\plug-ins\image
 \Program Files\AliasWavefront\Maya3.0\bin\plug-ins
 \Program Files\AliasWavefront\Maya3.0

ins\MayaWorldExport30.mll	\bin\plug-ins
\tools\Plugins\AliasWavefront\Maya3.0	\Program Files\AliasWavefront\Maya3.0
\scripts\LithTechModelExportOptions.mel	\scripts
\tools\Plugins\AliasWavefront\Maya3.0	\Program Files\AliasWavefront\Maya3.0
\scripts\LithTechWorldExportOptions.mel	\scripts
\tools\Plugins\AliasWavefront\Maya3.0	\Program Files\AliasWavefront\Maya3.0
\scripts\LithTechWorldImportOptions.mel	\scripts

3D Studio Max 6.0 SDK with Character Studio 4 SDK

<u>Plugin File</u>	<u>Install Path</u>
\tools\Plugins\3dsmax6\plugins\MaxDTXLoader60.bmi	\3dsmax6\plugins
\tools\Plugins\3dsmax6\plugins\MaxModelExport60.dle	\3dsmax6\plugins
\tools\Plugins\3dsmax6\plugins\MaxWorldExport60.dle	\3dsmax6\plugins
\tools\Plugins\3dsmax6\plugins\MaxWorldImport60.dli	\3dsmax6\plugins

3D Studio Max 5.0 SDK with Character Studio 4 SDK

<u>Plugin File</u>	<u>Install Path</u>
\tools\Plugins\3dsmax5\plugins\MaxDTXLoader50.bmi	\3dsmax5\plugins
\tools\Plugins\3dsmax5\plugins\MaxModelExport50.dle	\3dsmax5\plugins
\tools\Plugins\3dsmax5\plugins\MaxWorldExport50.dle	\3dsmax5\plugins
\tools\Plugins\3dsmax5\plugins\MaxWorldImport50.dli	\3dsmax5\plugins

3D Studio Max 4.0 SDK with Character Studio 3.1.2 SDK

<u>Plugin File</u>	<u>Install Path</u>
\tools\Plugins\3dsmax4\plugins\MaxDTXLoader40.bmi	\3dsmax4\plugins
\tools\Plugins\3dsmax4\plugins\MaxModelExport40.dle	\3dsmax4\plugins
\tools\Plugins\3dsmax4\plugins\MaxWorldExport40.dle	\3dsmax4\plugins
\tools\Plugins\3dsmax4\plugins\MaxWorldImport40.dli	\3dsmax4\plugins

3D Studio Max 3.0 SDK with Character Studio 3.0.2 SDK

<u>Plugin File</u>	<u>Install Path</u>
\tools\Plugins\3dsmax3\plugins\MaxDTXLoader30.bmi	\3DSMAX3\Plugins
\tools\Plugins\3dsmax3\plugins\MaxModelExport30.dle	\3DSMAX3\Plugins
\tools\Plugins\3dsmax3\plugins\MaxWorldExport30.dle	\3DSMAX3\Plugins
\tools\Plugins\3dsmax3\plugins\MaxWorldImport30.dli	\3DSMAX3\Plugins

Photoshop

<u>Plugin File</u>	<u>Install Path</u>
\tools\Plugins\DTXFormat.8bi	\Program Files\Adobe\Photoshop x\Plug-Ins\File Formats

Third Party SDK Installation

The Jupiter third party components should be installed to the following subdirectories of your Jupiter installation:

<u>Third Party SDK Name</u>	<u>Install Path</u>	<u>This SDK is needed for</u>
Maya 6.0 SDK	\Engine\tools\Plugins\sdkfiles\Maya60SDK\include\maya \Engine\tools\Plugins\sdkfiles\Maya60SDK\lib	Jupiter Enterprise
Maya 5.0 SDK	\Engine\tools\Plugins\sdkfiles\Maya50SDK\include\maya \Engine\tools\Plugins\sdkfiles\Maya50SDK\lib	Jupiter Enterprise

Maya 4.5 SDK	\\Engine\\tools\\Plugins\\sdkfiles\\Maya45SDK\\include\\maya \\Engine\\tools\\Plugins\\sdkfiles\\Maya45SDK\\lib	Jupiter Enterprise
Maya 4.0 SDK	\\Engine\\tools\\Plugins\\sdkfiles\\Maya40SDK\\include\\maya \\Engine\\tools\\Plugins\\sdkfiles\\Maya40SDK\\lib	Jupiter Enterprise
Maya 3.0 SDK	\\Engine\\tools\\Plugins\\sdkfiles\\Maya30SDK\\include\\maya \\Engine\\tools\\Plugins\\sdkfiles\\Maya30SDK\\lib	Jupiter Enterprise
3D Studio Max 6.0 SDK with Character Studio 4 SDK	\\Engine\\tools\\Plugins\\sdkfiles\\MAX60SDK\\cssdk\\include \\Engine\\tools\\Plugins\\sdkfiles\\MAX60SDK\\cssdk\\lib \\Engine\\tools\\Plugins\\sdkfiles\\MAX60SDK\\include \\Engine\\tools\\Plugins\\sdkfiles\\MAX60SDK\\lib	Jupiter Enterprise
3D Studio Max 5.0 SDK with Character Studio 4 SDK	\\Engine\\tools\\Plugins\\sdkfiles\\MAX50SDK\\cssdk\\include \\Engine\\tools\\Plugins\\sdkfiles\\MAX50SDK\\cssdk\\lib \\Engine\\tools\\Plugins\\sdkfiles\\MAX50SDK\\include \\Engine\\tools\\Plugins\\sdkfiles\\MAX50SDK\\lib	Jupiter Enterprise
3D Studio Max 4.0 SDK with Character Studio 3.1.2 SDK	\\Engine\\tools\\Plugins\\sdkfiles\\MAX40SDK\\CSTUDIO\\SDK \\Engine\\tools\\Plugins\\sdkfiles\\MAX40SDK\\Include \\Engine\\tools\\Plugins\\sdkfiles\\MAX40SDK\\lib	Jupiter Enterprise
3D Studio Max 3.0 SDK with Character Studio 3.0.2 SDK	\\Engine\\tools\\Plugins\\sdkfiles\\MAX30SDK\\CSTUDIO\\SDK \\Engine\\tools\\Plugins\\sdkfiles\\MAX30SDK\\Include \\Engine\\tools\\Plugins\\sdkfiles\\MAX30SDK\\lib	Jupiter Enterprise

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



Samples

The samples are organized into the following directories:

- [Sample Base](#)
- [Art Content Samples](#)
 - [Audio Samples](#)
 - [Graphics Samples](#)
- [Networking Samples](#)
- [Samples Using Models](#)
- [Samples Using Objects](#)

Base

The \\Samples\\base\\mpflycam folder contains the **MPFlyCam** sample. This sample is the same as SampleBase but has been expanded to include the ability to play in multi-player server mode.

The \\Samples\\base\\samplebase folder contains the **SampleBase** sample. This simple example helps you get started with the Jupiter System. This sample also shows how to use the networking system.

The \\Samples\\base\\simplephys folder contains the **SimplePhys** sample. This sample is the same as the MpFlyCam sample, but has been expanded to include physics such as gravity and stair-stepping.

Content

The \\Samples\\content\\maya\\ folder contains sample content from No One Lives Forever™ 2 that can aid content creators in creating artwork that is easily usable with the Jupiter System. The No One Lives Forever™ 2 team uses Maya for character models. Included are samples of character and player view models that are properly rigged to work with the Jupiter System. These models should be exported to ModelEdit using the Maya 4.0 model exporter.

The \\Samples\\content\\maya\\SealHunter folder contains some of the artwork used to create the Seal Hunter sample mini game.

These models were created using Maya 4.0.

Audio

The \Samples\audio\music folder contains the **Music** sample. This sample demonstrates Jupiter's DirectMusic interface using music files from No One Lives Forever™ 2.

The \Samples\audio\sounds folder contains the **Sounds** sample. This sample allows developers to experiment with different sound settings for multiple sounds.

Graphics

The \Samples\graphics\clientfx folder contains the **ClientFX** sample. This sample allows developers to browse and view effects created by artists. This sample reads through all of the effects in the current .fxf file and creates a graphical selection menu allowing you to scroll through and play any effects in that list.

The \Samples\graphics\drawprim\ folder contains the **DrawPrim** sample. This sample demonstrates the use of the DrawPrim interface as a means for creating procedural terrain. The height map data in this sample was created from USGS data.

The \Samples\graphics\fonts\ folder contains the **Fonts** sample. This sample demonstrates the creation and display of vector (*.ttf) and bitmap (*.dtx) fonts.

The \Samples\graphics\renderdemo\ folder contains the **RenderDemo** sample. This sample demonstrates basic polygon throughput. Models can be added in rows and columns while the polygon count and frame rate are displayed.

The \Samples\graphics\shaders\ folder contains the **Shaders** sample. This sample demonstrates the use of vertex and pixels shaders on models using RenderStyles.

The \Samples\graphics\specialeffects1 folder contains the **SpecialEffects1** sample. This sample combines the effects of the Animations2 tutorial and the ClientFX sample.

The \Samples\graphics\video folder contains the **Video** sample. This sample demonstrates the video playing functionality of Jupiter.

Networking

The \Samples\networking\netest folder contains the **NetTest** sample. This sample demonstrates a simple way to create an in-game LAN game browser.

The \Samples\networking\sealhunter folder contains the **Seal Hunter** sample. This light humored sample implements a single player and networked multiplayer mini game with the following features:

- Single player or multiplayer game (supports up to 12 players in multiplayer)
 - Windows and Linux stand-alone server for hosting games
 - Scoring system, current score HUD, and ranking screen
 - Multiplayer chat
- The world features environment mapping, translucency, dynamic lighting, shadows, and skybox
 - ClientFX particle systems, sprites, and polytrails are used for the effects
 - The character can attach 3 different weapons in multiplayer
 - Audio clips played randomly for player smack talk
 - Simple AI and collision for seals

Models

The \Samples\models\animations folder contains the **Animations** sample. This sample is the same as the Projectiles sample, but has been expanded to include code implementing simple model animations in a multiplayer environment.

The \Samples\models\animations2 folder contains the **Animations2** sample. This sample is the same as the Animations sample, but has been expanded to include code implementing more advanced model animations in a multiplayer environment.

The \Samples\models\attachments folder contains the **Attachments** sample. This sample builds on the SampleBase sample and shows how to make an attachment, break an attachment, and attach different objects to one socket.

Objects

The \Samples\objects\doors folder contains the **Doors** sample. This sample shows how to create a simple door that opens when touched by the player or a projectile. It also allows placement of the door through the DEdit application.

The \Samples\objects\pickups folder contains the **Pickups** sample. This sample is the same as the SimplePhys sample, but has been expanded to include a box pickup. This tutorial shows how to create and implement code for pickups and powerups.

The \Samples\objects\projectiles folder contains the **Projectiles** sample. This sample is the same as the Pickups sample, but has been expanded to include the ability to throw an object. This tutorial shows how to create and implement code for

projectiles.

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



Tutorials

This section lists all of the tutorials available for your use. The location of tutorials vary. Some tutorials are specific to tasks performed in ModelEdit, DEdit, or FXEd. Other tutorials combine tasks from multiple tools, and may also include programming steps.

The following tutorials are available for the Jupiter system:

Art and Level Tutorials

[Adding an Attachment to Your Model](#)

Provides instruction on adding an attachment to your model in Jupiter.

[Adding Sky to Your Level](#)

Provides instruction on adding a sky box.

[Creating Animations](#)

Provides instructions for programmatically playing simple model animations in multiplayer environment.

[Building Your First World](#)

Provides instruction on creating a simple world.

[Creating a Cut Scene](#)

Provides instruction on using ModelEdit, DEdit, and FXEd to create a cinematic cut scene in your game.

[Lighting Your World](#)

Provides instructions on selecting and applying different types of lighting.

[Using Brushes](#)

Provides instruction on using brushes.

[Working With AI](#)

Provides instructions on using the AI provided in the NOLF 2 game code.

Programming Tutorials

[Multiplayer Flying Camera](#)

Provides a tutorial on adding multiplayer capabilities to the [Sample Base](#) sample.

[Simple Physics](#)

Provides information about the problems and solutions that arise with physics in a multiplayer environment, and builds on the [Flying Camera](#) tutorial as a base.

[Creating Pickups](#)

Provides instructions on creating objects in the game that characters can pick up and use. Builds on the [Simple Physics](#) tutorial.

[Creating Projectiles](#)

Provides instructions on creating moving objects in the game (such as bullets) that can connect and interact with models and world geometry.

[Creating Animations](#)

Provides instructions on creating simple model animations in a multi-player environment, and builds on the [Creating Projectiles](#) tutorial.

[Creating Animations 2](#)

Provides instructions on creating more advanced model animations in a multi-player environment and builds on the [Creating Animations](#) tutorial.

[Creating Doors Tutorial](#)

Provides instructions on creating a simple door that opens when touched by the player or a projectile.

[Creating Special Effects 1 Tutorial](#)

Provides instructions on creating special effects and combines the [Animations2](#) tutorial and the [clientfx](#) tutorial.

[Top](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

Content Guide



Support

For assistance with technical issues relating to the Jupiter System, e-mail Touchdown Entertainment support at support@touchdownentertainment.com.

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.

Shortcut Text	Internet Address
	http://www.touchdownentertainment.com/jupiter.htm
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: Intro\Intro.htm
Windows Requirements	file:///C:/Users/khara/AppData/Local/Temp/~hh7EAB.htm#Windows
Required Environment Variables	file:///C:/Users/khara/AppData/Local/Temp/~hh7EAB.htm#EnvVariables
http://www.radgametools.com/	http://www.radgametools.com/
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: Intro\system_requirements.htm
Learn the Jupiter System	file:///C:/Users/khara/AppData/Local/Temp/~hh7EAB.htm#UnderstandJupiter
Review your Design Documentation	file:///C:/Users/khara/AppData/Local/Temp/~hh7EAB.htm#ReviewDesignDoc
Identify your Development Tasks	file:///C:/Users/khara/AppData/Local/Temp/~hh7EAB.htm#IdentifyDevelopmentTasks
Modify your Object Properties	file:///C:/Users/khara/AppData/Local/Temp/~hh7EAB.htm#ModifyObjectProperties
Create your New Levels	file:///C:/Users/khara/AppData/Local/Temp/~hh7EAB.htm#CreateNewLevels
Develop your Artwork	file:///C:/Users/khara/AppData/Local/Temp/~hh7EAB.htm#DevelopArtwork
Top	file:///C:/Users/khara/AppData/Local/Temp/~hh7EAB.htm#top
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: ImpGame\ImpGame.htm
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: Intro\components.htm
Building from the Command Line	file:///C:/Users/khara/AppData/Local/Temp/~hh7EAB.htm#BuildingfromtheCommandLine
Building from Visual Studio	file:///C:/Users/khara/AppData/Local/Temp/~hh7EAB.htm#BuildingfromVisualStudio
Running the Samples	file:///C:/Users/khara/AppData/Local/Temp/~hh7EAB.htm#RunningtheSamples
Running No One Lives Forever™ 2	file:///C:/Users/khara/AppData/Local/Temp/~hh7EAB.htm#RunningTheGame
Debugging the Samples	file:///C:/Users/khara/AppData/Local/Temp/~hh7EAB.htm#DebuggingtheSamples
Launching the Game Within DEdit	file:///C:/Users/khara/AppData/Local/Temp/~hh7EAB.htm#LaunchingTheGameWithinDEdit
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: Intro\building_running.htm
Content Plugins Installation	file:///C:/Users/khara/AppData/Local/Temp/~hh7EAB.htm#Plugins
Third Party SDK Installation	file:///C:/Users/khara/AppData/Local/Temp/~hh7EAB.htm#ThirdParty
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: Intro\content_plugins.htm
Sample Base	file:///C:/Users/khara/AppData/Local/Temp/~hh7EAB.htm#Base
Art Content Samples	file:///C:/Users/khara/AppData/Local/Temp/~hh7EAB.htm#Content
Audio Samples	file:///C:/Users/khara/AppData/Local/Temp/~hh7EAB.htm#Audio
Graphics Samples	file:///C:/Users/khara/AppData/Local/Temp/~hh7EAB.htm#Graphics
Networking Samples	file:///C:/Users/khara/AppData/Local/Temp/~hh7EAB.htm#Networking
Samples Using Models	file:///C:/Users/khara/AppData/Local/Temp/~hh7EAB.htm#Models
Samples Using Objects	file:///C:/Users/khara/AppData/Local/Temp/~hh7EAB.htm#Objects
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: Intro\samples.htm
Adding an Attachment to Your Model	file:///C:/Users/khara/AppData/Local/Temp/TAddAtch.htm
Adding Sky to Your Level	file:///C:/Users/khara/AppData/Local/Dedit/TAddSky.htm
Creating Animations	file:///C:/Users/khara/AppData/Local/Temp/animat.htm
Building Your First World	file:///C:/Users/khara/AppData/Local/Dedit/TBldWrld.htm
Creating a Cut Scene	file:///C:/Users/khara/AppData/Local/Temp/cutscene.htm
Lighting Your World	file:///C:/Users/khara/AppData/Local/Temp/WrldLitn.htm
Using Brushes	file:///C:/Users/khara/AppData/Local/Dedit/Brushes.htm
Working With AI	file:///C:/Users/khara/AppData/Local/WAI/mAI.htm
Multiplayer Flying Camera	file:///C:/Users/khara/AppData/Local/ProgGuid/Prog/FlyCam.htm
Sample Base	file:///C:/Users/khara/AppData/Local/ProgGuid/Prog/SampleB.htm
Simple Physics	file:///C:/Users/khara/AppData/Local/ProgGuid/Prog/SimpPhys.htm
Creating Pickups	file:///C:/Users/khara/AppData/Local/ProgGuid/Prog/PickUps.htm
Simple Physics	file:///C:/Users/khara/AppData/Local/ProgGuid/Prog/SimpPhys.html
Creating Projectiles	file:///C:/Users/khara/AppData/Local/ProgGuid/Prog/Bullets.htm
Creating Animations 2	file:///C:/Users/khara/AppData/Local/Temp/animat2.htm

Shortcut Text	Internet Address
Creating Doors Tutorial	file:///C:/Users/khara/AppData/Local/Temp/Doors1.htm
Creating Special Effects 1 Tutorial	file:///C:/Users/khara/AppData/Local/Temp/SP_FX1.htm
clientfx	file:///C:/Users/khara/AppData/Local/ProgGuid/ClientFX/mCliFX.htm
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: Tutorials\mTutor.htm
support@touchdownentertainment.com	mailto:support@touchdownentertainment.com
Send feedback regarding this page.	mailto:support@touchdownentertainment.com?subject=JupiterDevGuide Feedback: Intro\support.htm